

Laboratório de desenvolvimento de software

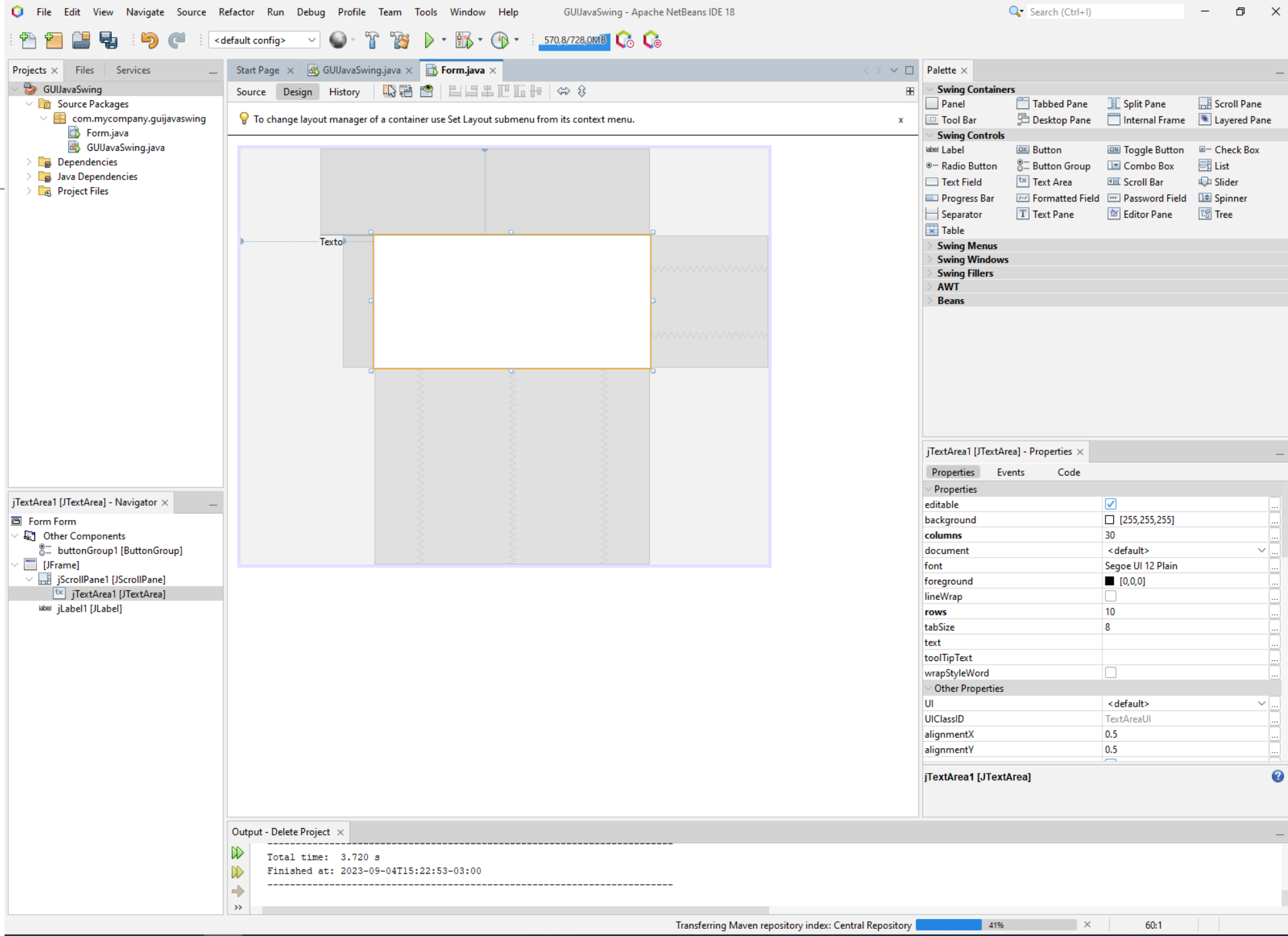
Java Swing

Java Swing

- Na aula de hoje, vamos trabalhar com outros componentes:
 - Text Area
 - Radio Button
 - CheckBox
 - ComboBox
 - Table

Text Area

- Adicione um JFrame Form
- E dentro dele, adicione um label e um text area



FileEditViewNavigateSourceRefactorRunDebugProfileTeamToolsWindowHelp

GUIJavaSwing - Apache NetBeans IDE 18

Search (Ctrl+I)

ProjectsFilesServices

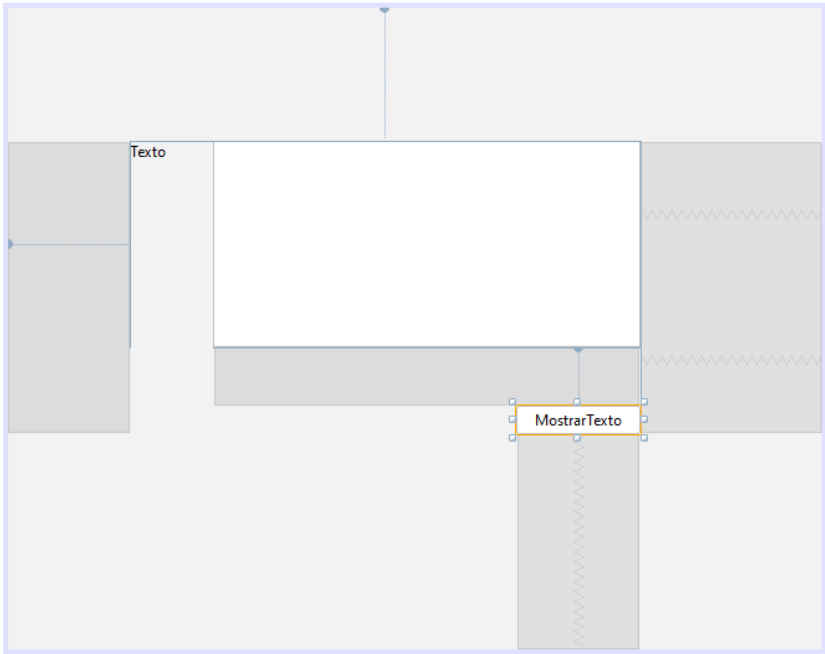
GUIJavaSwing

- Source Packages
 - com.mycompany.guijavaswing
 - Form.java
 - GUIJavaSwing.java
 - Dependencies
 - Java Dependencies
 - Project Files

Start PageGUIJavaSwing.javaForm.java

SourceDesignHistory

To add a component multiple times, select it via click in palette and then Shift-click on design canvas.



jButton1 [JButton] - Navigator

Form Form

- Other Components
- buttonGroup1 [ButtonGroup]
- JFrame
 - jScrollPane1 [JScrollPane]
 - jTextArea1 [JTextArea]
 - jLabel1 [JLabel]
 - jButton1 [JButton]

Palette

Swing Containers

- Panel
- Tabbed Pane
- Split Pane
- Scroll Pane
- ToolBar
- Desktop Pane
- Internal Frame
- Layered Pane

Swing Controls

- Label Label
- Button
- ToggleButton
- CheckBox
- Radio Button
- Button Group
- Combo Box
- List
- Text Field
- Text Area
- Scroll Bar
- Slider
- Progress Bar
- Formatted Field
- Password Field
- Spinner
- Separator
- Text Pane
- Editor Pane
- Tree
- Table

Swing Menus

Swing Windows

Swing Fillers

AWT

Beans

jButton1 [JButton] - Properties

PropertiesEventsCode

Properties	
action	<none>
background	[255,255,255]
font	Segoe UI 12 Plain
foreground	[0,0,0]
icon	<none>
mnemonic	
text	MostrarTexto
toolTipText	
Other Properties	
UIClassID	ButtonUI
actionCommand	btnMostrarTexto
alignmentX	0.0
alignmentY	0.5
autoscrolls	
baselineResizeBehavior	CENTER_OFFSET
border	[FlatButtonBorder]
borderPainted	☒
actionCommand (java.lang.String) actionCommand	

Output - Delete Project

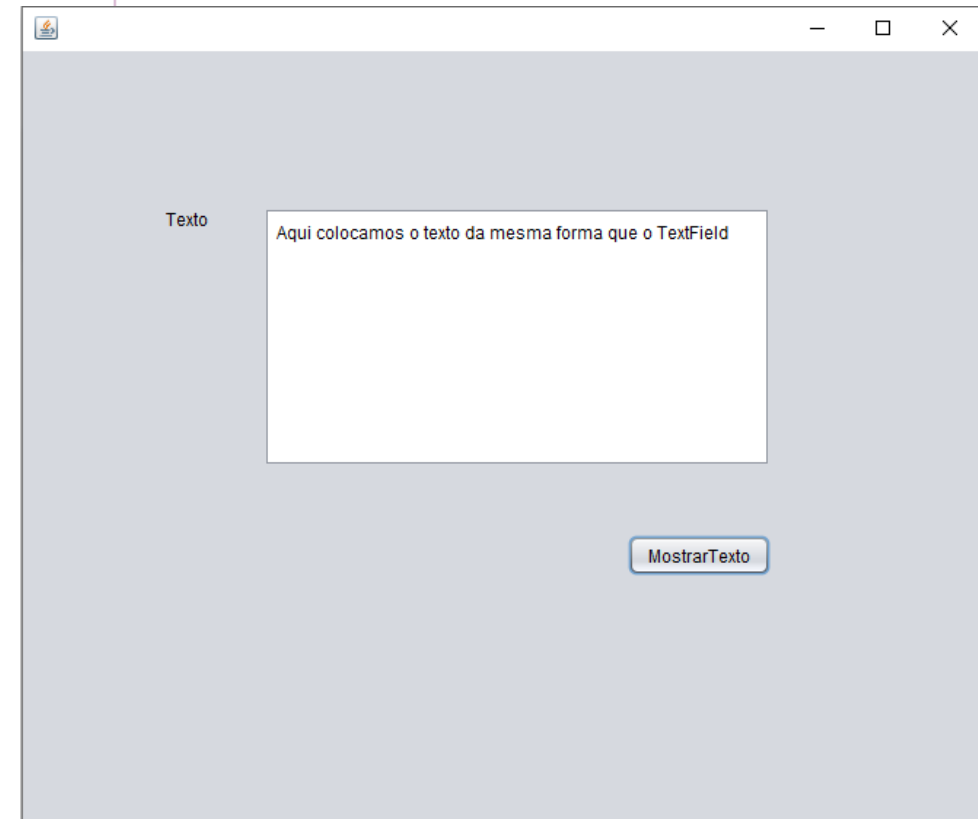
Total time: 3.720 s

Finished at: 2023-09-04T15:22:53-03:00

Transferring Maven repository index: Central Repository 42%60:1

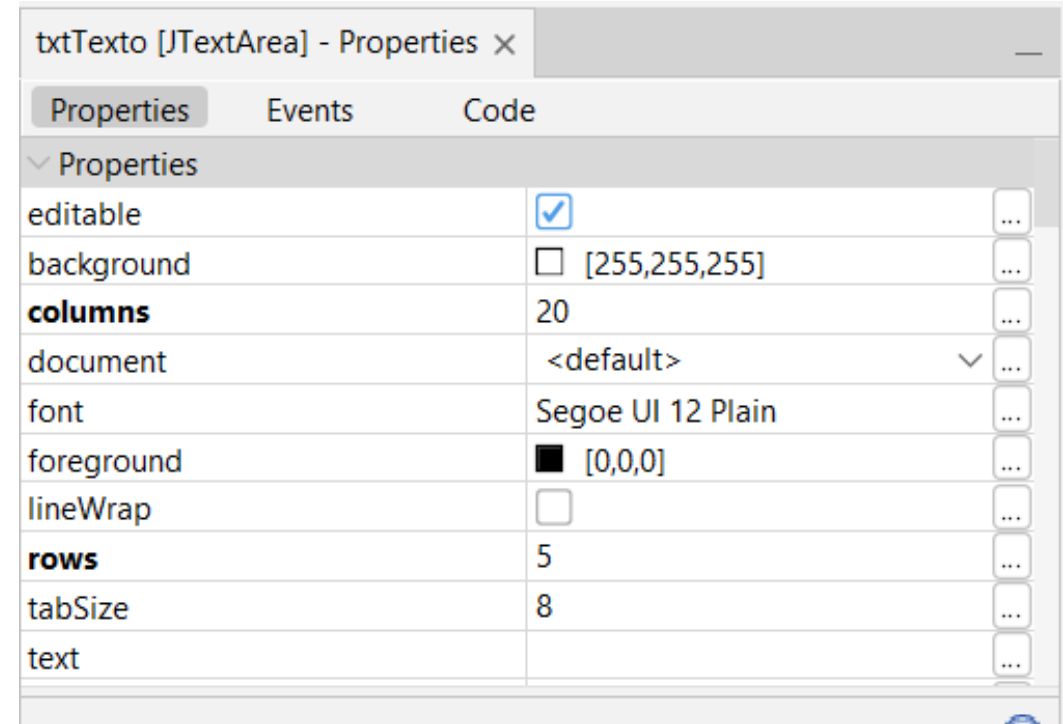
Text Area

```
private void btnMostraTextoActionPerformed(java.awt.event.ActionEvent evt) {  
    txaTexto.setText("Aqui colocamos o texto da mesma forma que o TextField");  
}
```



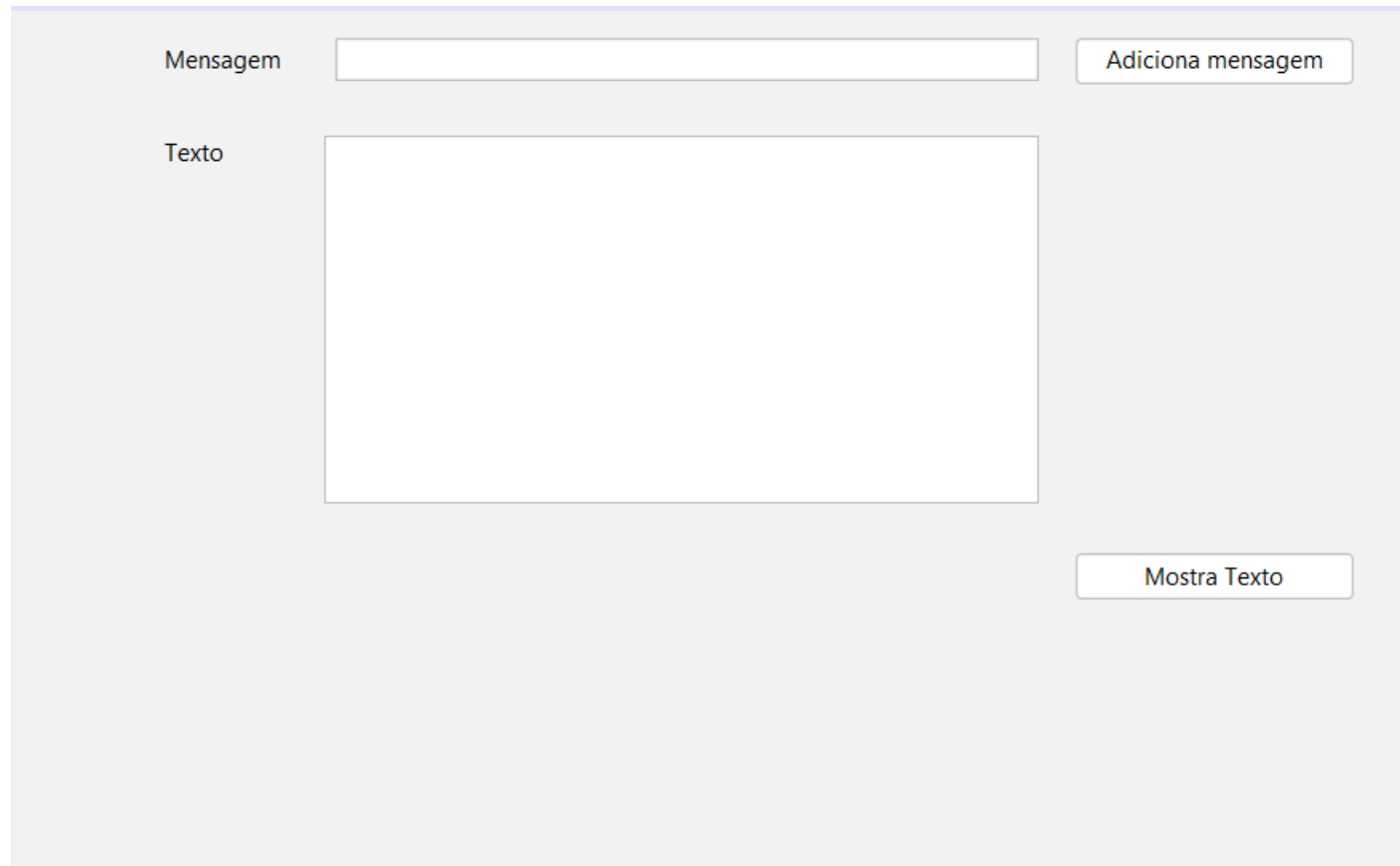
Text Area

- A diferença do text field é que aqui conseguimos mudar a quantidade de colunas e linhas e ir adicionando textos dentro dele.



Text Area

- Agora adicione um text field e outro botão



The image shows a web form with a light gray background. It contains the following elements:

- A label "Mensagem" followed by a single-line text input field.
- A button labeled "Adiciona mensagem" located to the right of the "Mensagem" input field.
- A label "Texto" followed by a large multi-line text area.
- A button labeled "Mostra Texto" located at the bottom right of the form.

Text Area

- Adicione o código abaixo na ação do botão adicionado

```
private void btnAddMsgActionPerformed(java.awt.event.ActionEvent evt) {  
    txaTexto.append(str: txtMensagem.getText());  
}
```

Text Area

- E como ficou? Para melhorar, podemos limpar o textfield e quebrar a linha após cada mensagem adicionada.

```
private void btnAddMsgActionPerformed(java.awt.event.ActionEvent evt) {  
    txaTexto.append(txtMensagem.getText()+"\n");  
    txtMensagem.setText("");  
}
```

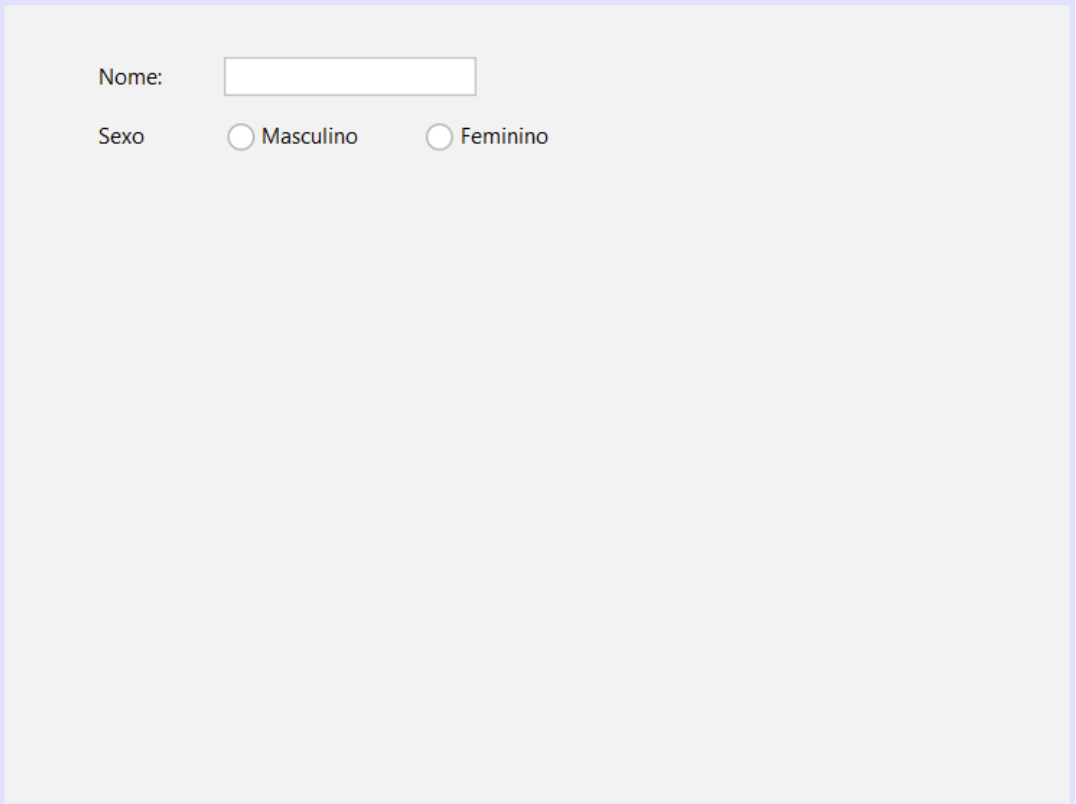
- Entenderam para que funciona o método append?

Radion Button

- É usado para permitir que os usuários escolham uma única opção de um conjunto pré-definido de opções mutuamente exclusivas.
- Eles são chamados de "radio buttons" porque se assemelham aos botões de rádio físicos em um rádio antigo, onde você só pode selecionar uma estação de rádio por vez.

Radio Button

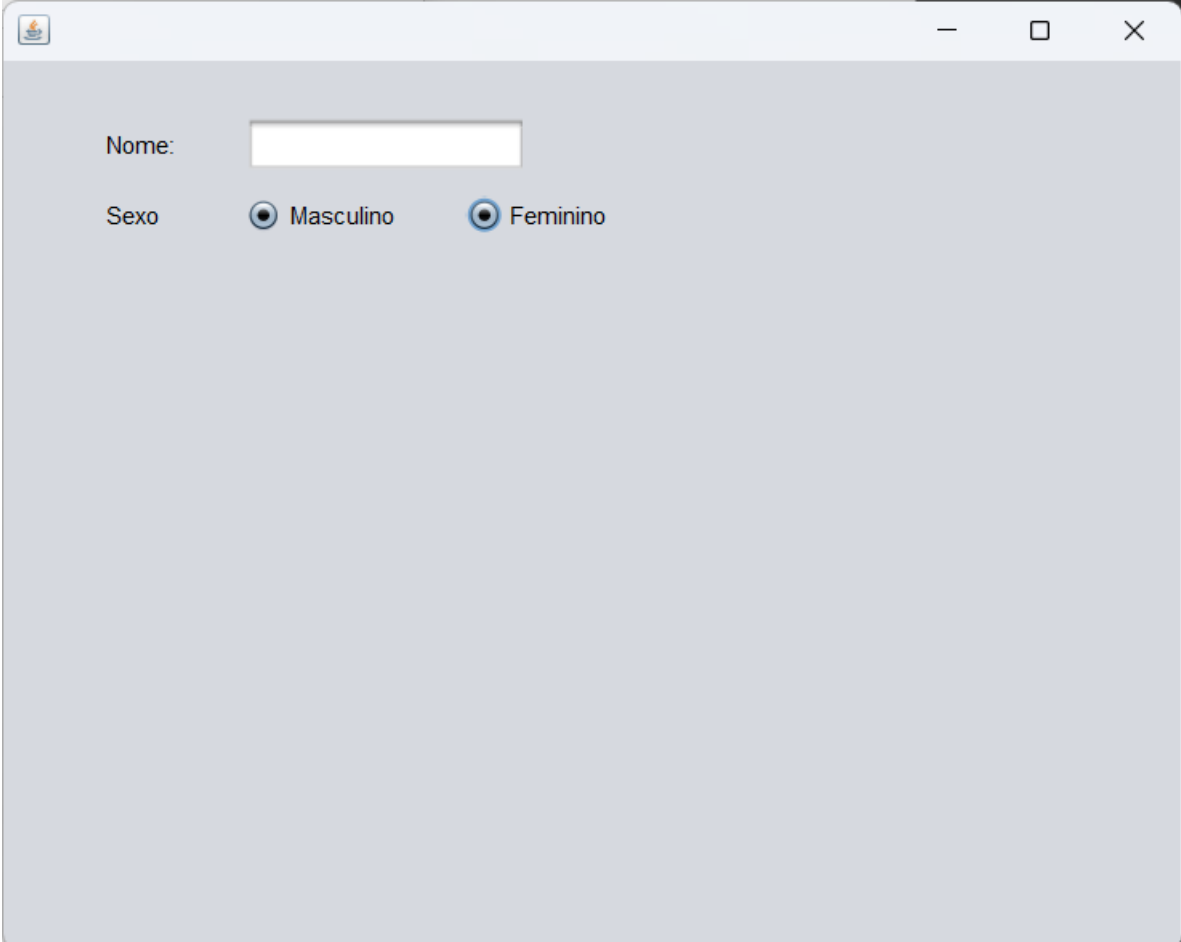
- Adicione um novo JForm e adicione um text field dois Radion Button e compile o código..



A screenshot of a Java Swing window with a light gray background. It contains two labels: 'Nome:' and 'Sexo'. The 'Nome:' label is positioned to the left of a white text field with a thin black border. The 'Sexo' label is positioned to the left of two radio buttons. The first radio button is selected and is followed by the text 'Masculino'. The second radio button is unselected and is followed by the text 'Feminino'.

Radio Button

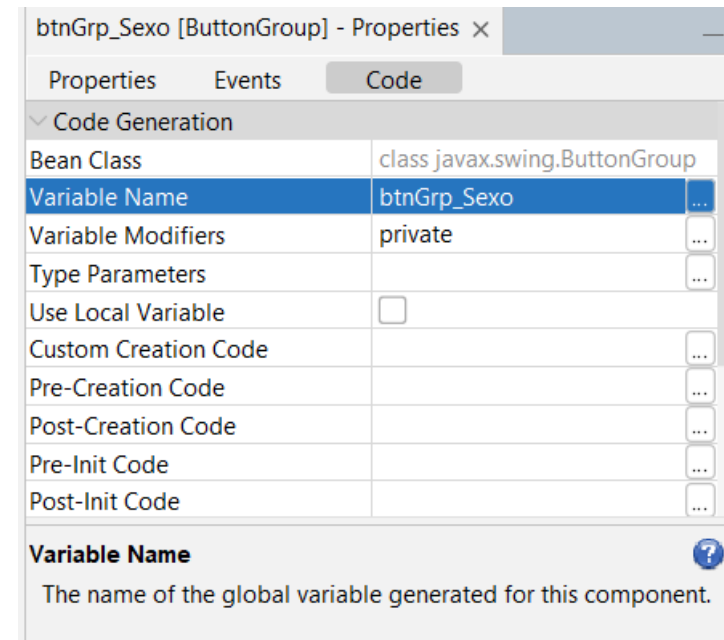
- Funcionou? Este botão não deve possibilitar selecionar os dois
- Como resolver?



A screenshot of a web application window. The window has a title bar with a small icon on the left and standard minimize, maximize, and close buttons on the right. The main content area is light gray and contains two form elements. The first is a label 'Nome:' followed by a white text input field. The second is a label 'Sexo' followed by two radio button options: 'Masculino' and 'Feminino'. Both radio buttons are selected, indicated by a blue dot in the center of each button's circle.

Radio Button

- Adicione um Button Group
- Ele não aparecerá no formulário



Button Group

- Selecione o RadioButton e vá em propriedades

rdo_Masculino [JRadioButton] - Properties ×

PropertiesEventsCode

▼ Properties

action	<none>	▼	...
background	 [242,242,242]		...
model	<default>	▼	...
buttonGroup	<none>	▼	...
font	Segoe UI 12 Plain		...
foreground	 [0,0,0]		...
mnemonic			...
selected	☐		...
text	Masculino		...
toolTipText			...

background
(java.awt.Color) The background color of the component.

rdo_Masculino [JRadioButton] - Properties ×

PropertiesEventsCode

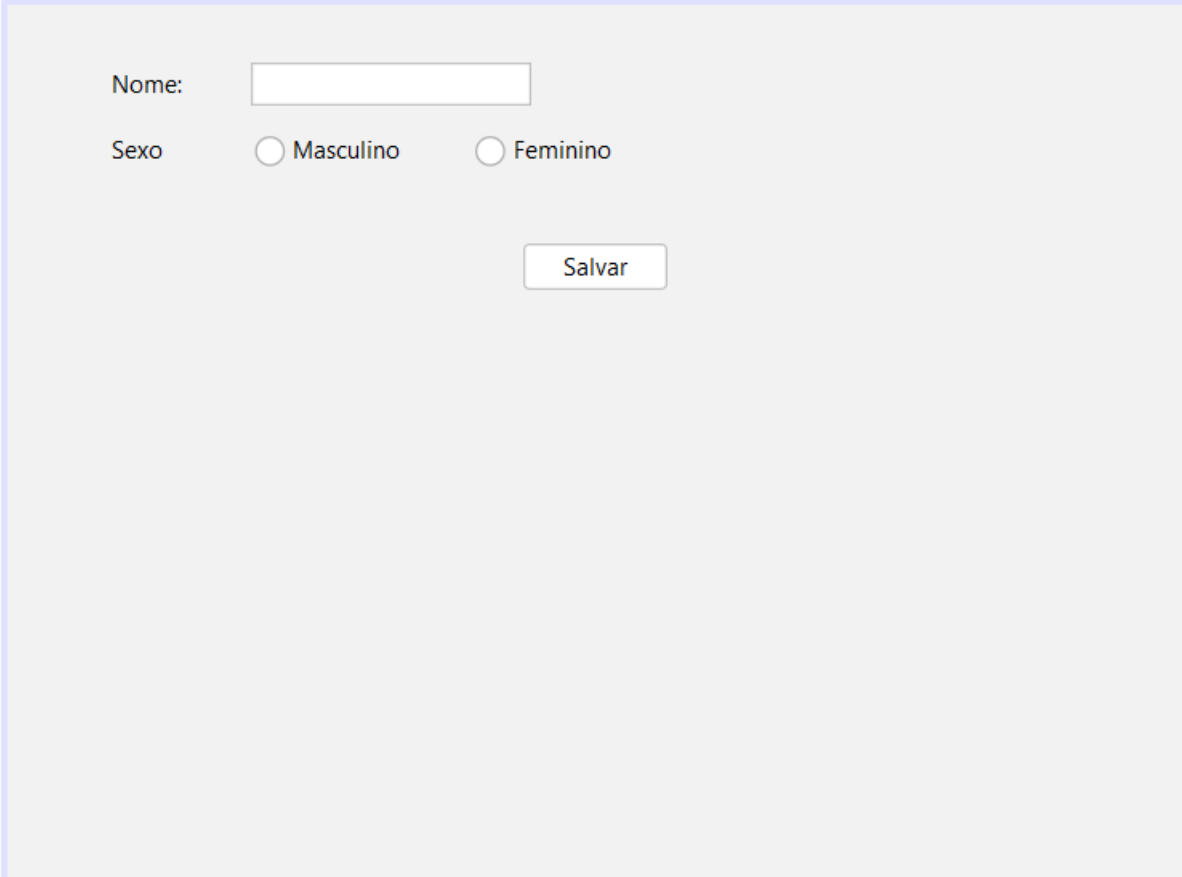
▼ Properties

action	<none>	▼	...
background	 [242,242,242]		...
model	<default>	▼	...
buttonGroup	btnGrp_Sexo	▼	...
font	Segoe UI 12 Plain		...
foreground	 [0,0,0]		...
mnemonic			...
selected	☐		...
text	Masculino		...
toolTipText			...

rdo_Masculino [JRadioButton]

Radio Button

- E como verificar qual está selecionado?
- Adicione um botão Salvar



Nome:

Sexo ☐ Masculino ☐ Feminino

Radio Button

- Declare uma variável global do tipo char chamada sexo
- Na ação do botão Salvar, adicione o código abaixo:

```
if(rdo_Masculino.isSelected()){
    sexo = 'M';
    JOptionPane.showMessageDialog(parentComponent: null,
        "Sexo selecionado: "+sexo, title: "Alerta!" , messageType: JOptionPane.OK_OPTION);
}
else if(rdo_Feminino.isSelected()){
    sexo = 'F';
    JOptionPane.showMessageDialog(parentComponent: null,
        "Sexo selecionado: "+sexo, title: "Alerta!" , messageType: JOptionPane.OK_OPTION);
}
else{
    JOptionPane.showMessageDialog(parentComponent: null,
        message: "Selecione um sexo", title: "Erro!" , messageType: JOptionPane.ERROR_MESSAGE);
}
```

Radio Button

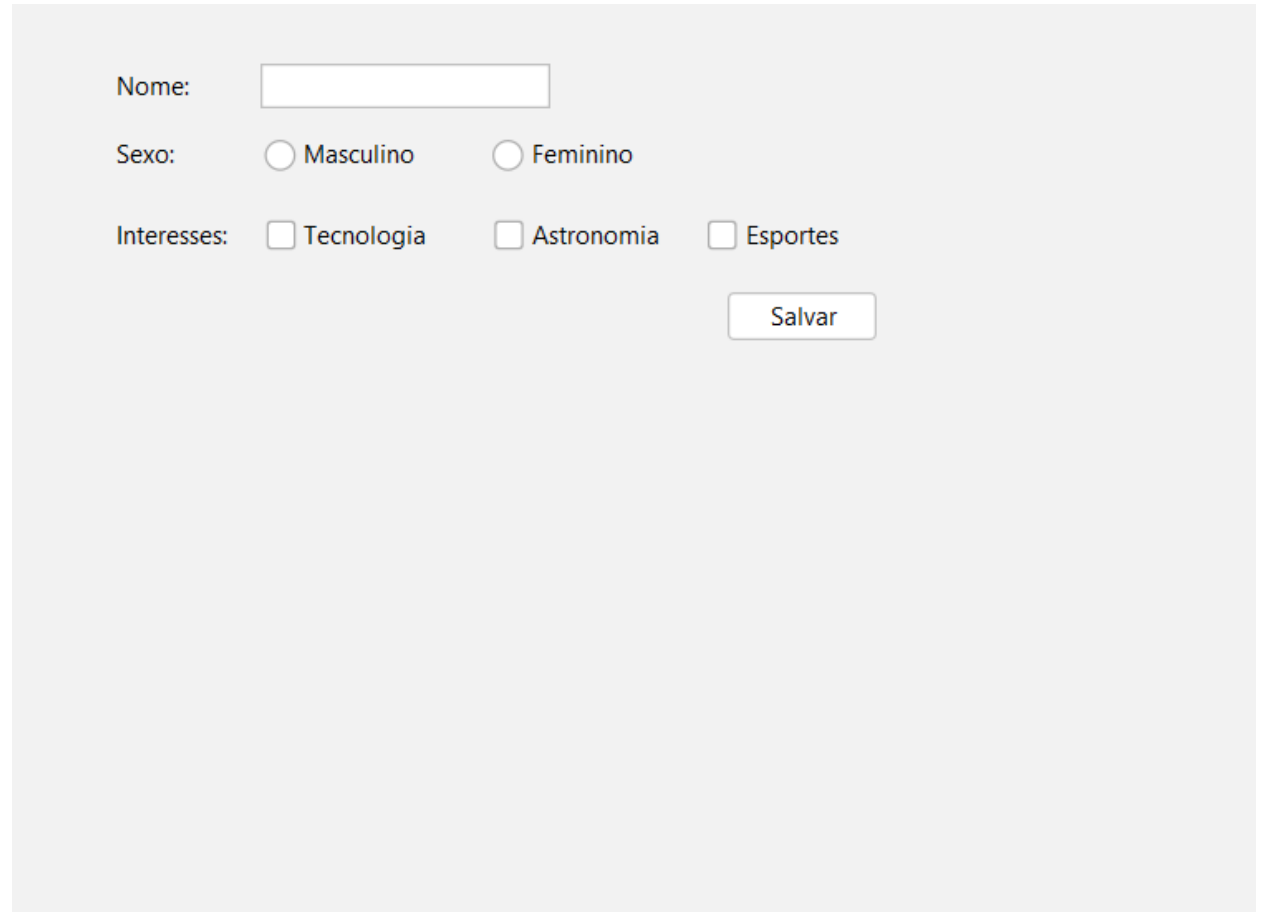
```
if(rdo_Masculino.isSelected()){
    sexo = 'M';
    JOptionPane.showMessageDialog(parentComponent: null,
        "Sexo selecionado: "+sexo, title: "Alerta!" , messageType: JOptionPane.OK_OPTION);
}
else if(rdo_Feminino.isSelected()){
    sexo = 'F';
    JOptionPane.showMessageDialog(parentComponent: null,
        "Sexo selecionado: "+sexo, title: "Alerta!" , messageType: JOptionPane.OK_OPTION);
}
else{
    JOptionPane.showMessageDialog(parentComponent: null,
        message: "Selecione um sexo", title: "Erro!" , messageType: JOptionPane.ERROR_MESSAGE);
}
```

Check Box

- Agora, vamos trabalhar com Check Box
- Permite aos usuários selecionar ou desmarcar uma ou mais opções de um conjunto de opções independentes.
- Os checkboxes são frequentemente usados quando você deseja que os usuários tenham a capacidade de escolher várias opções dentre várias disponíveis.

Check Box

- Adicione estes check box conforme a imagem, renomeie conforme o padrão visto em aula
- `chk_Tecnologia`, por exemplo



Nome:

Sexo: ☐ Masculino ☐ Feminino

Interesses: ☐ Tecnologia ☐ Astronomia ☐ Esportes

Check Box

- Declare três atributos do tipo bool

```
char sexo;  
boolean tecnologia, astronomia, esportes;
```

- E na ação do botão:

```
if(chk_Tecnologia.isSelected()){  
    tecnologia = true;  
}  
else{  
    tecnologia = false;  
}  
if(chk_Astronomia.isSelected()){  
    astronomia = true;  
}  
else{  
    astronomia = false;  
}  
if(chk_Esportes.isSelected()){  
    esportes = true;  
}  
else{  
    esportes = false;  
}  
JOptionPane.showMessageDialog(parentComponent: null, "Tecnologia: "+tecnologia+"\nAstronomia: "+astronomia+  
    "\nEsportes: "+esportes, title: "Alerta", messageType: JOptionPane.INFORMATION_MESSAGE);
```

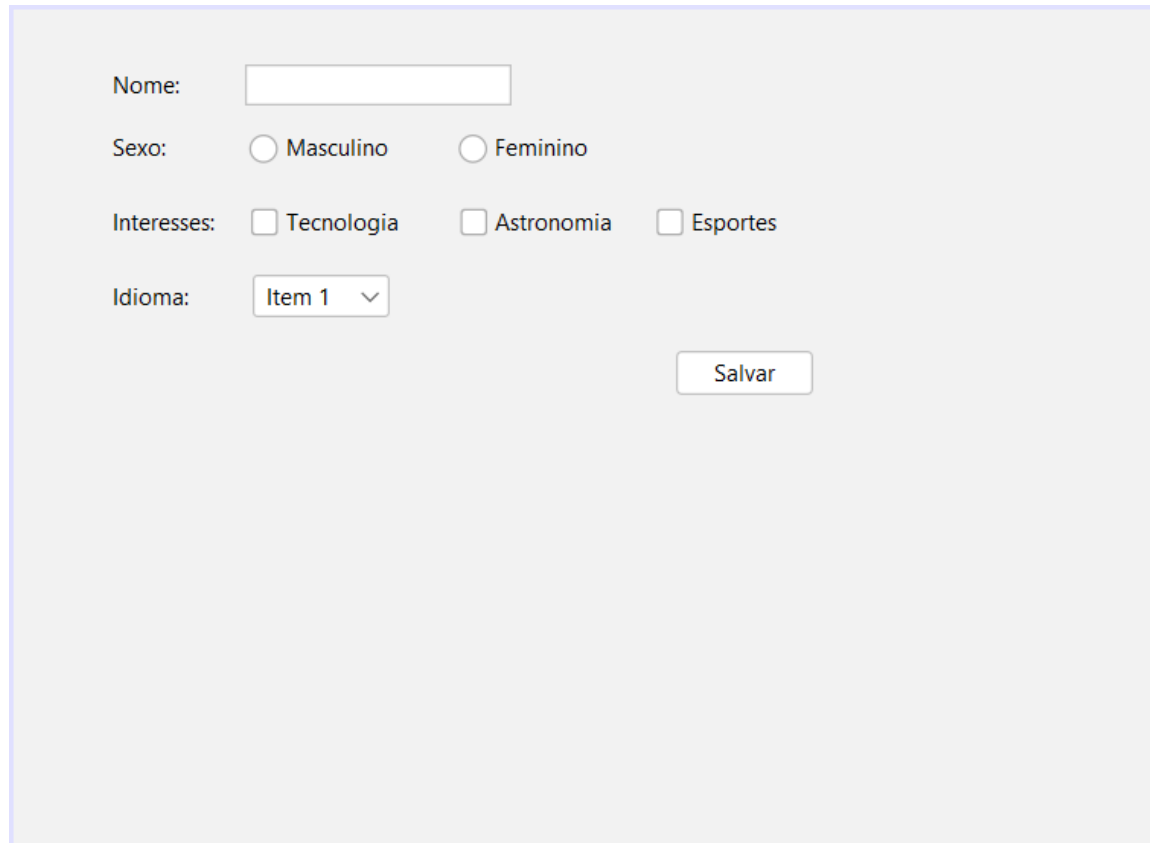
```
if(chk_Tecnologia.isSelected()){
    tecnologia = true;
}
else{
    tecnologia = false;
}
if(chk_Astronomia.isSelected()){
    astronomia = true;
}
else{
    astronomia = false;
}
if(chk_Esportes.isSelected()){
    esportes = true;
}
else{
    esportes = false;
}
JOptionPane.showMessageDialog(parentComponent: null, "Tecnologia: "+tecnologia+"\nAstronomia: "+astronomia+
    "\nEsportes: "+esportes, title: "Alerta", messageType: JOptionPane.INFORMATION_MESSAGE);
```

Combo Box

- Agora, vamos trabalhar com Combo Box
- Combina uma caixa de texto editável com uma lista suspensa de opções selecionáveis.
- São usados para permitir que os usuários escolham uma opção entre várias em um menu suspenso ou insiram um valor personalizado por meio da caixa de texto.
- Eles são úteis quando você deseja oferecer uma lista de opções predefinidas ou permitir que os usuários insiram dados personalizados.

Combo Box

- Adicione um Combo box que vai ser possível selecionar um idioma



Nome:

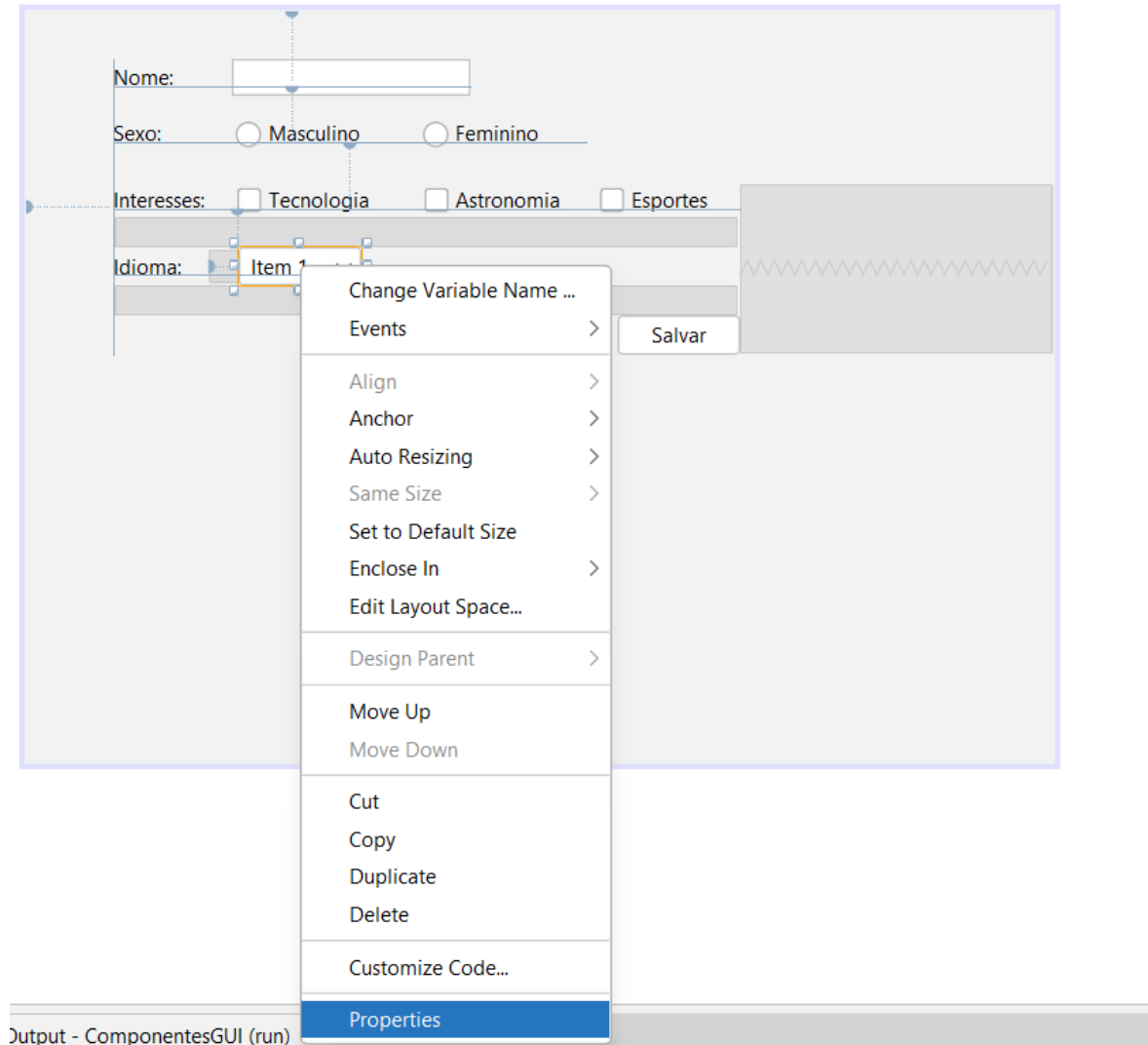
Sexo: ☐ Masculino ☐ Feminino

Interesses: ☐ Tecnologia ☐ Astronomia ☐ Esportes

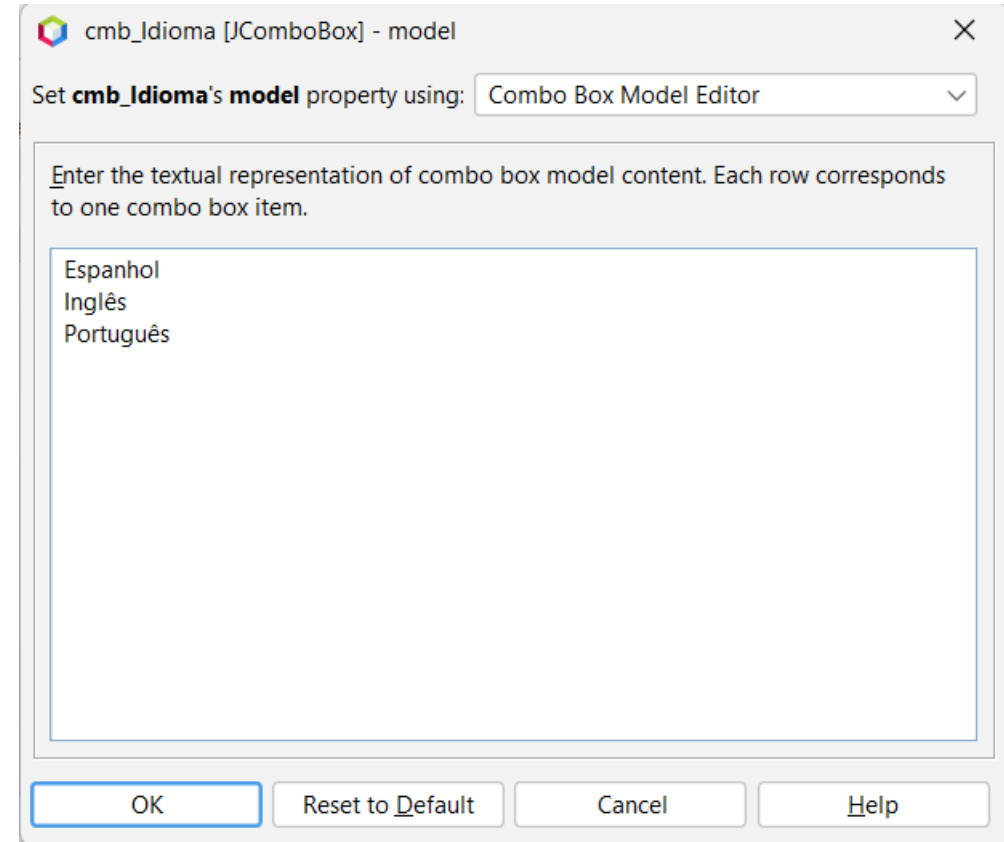
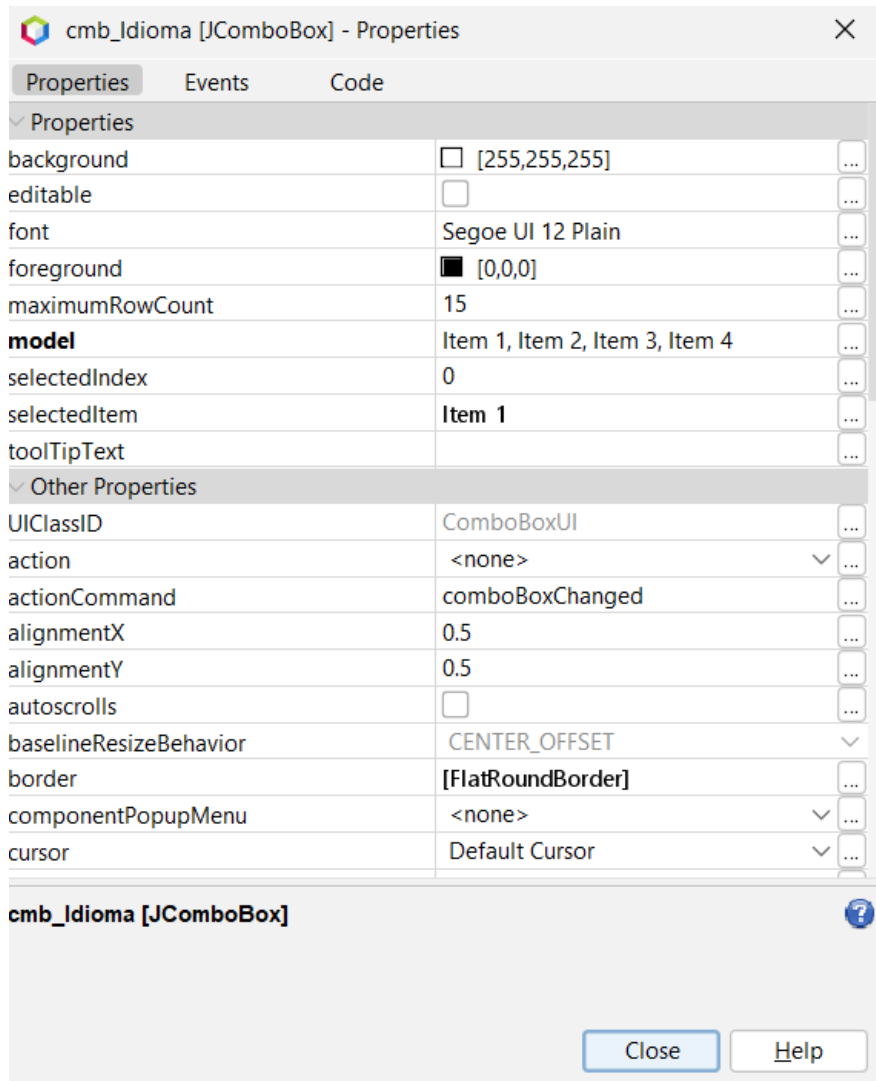
Idioma:

Salvar

Combo Box



Combo Box



Combo Box

```
JOptionPane.showMessageDialog(parentComponent: null,  
    "Idioma seleccionado:" +cmb_Idioma.getSelectedItem(),  
    title: "Alerta",messageType: JOptionPane.INFORMATION_MESSAGE);
```

Combo Box

- Agora, vamos criar uma classe

```
public class Pessoa {  
    public String nome;  
    public char sexo;  
    public String idioma;  
  
    public Pessoa(String nome, char sexo, String idioma) {  
        this.nome = nome;  
        this.sexo = sexo;  
        this.idioma = idioma;  
    }  
}
```

Combo Box

- Na ação do botão Salvar, adicione o código:

```
Pessoa p = new Pessoa(nome: txt_Nome.getText(), sexo, (String) cmb_Idioma.getSelectedItem());
```

```
15 public class Arquivo {
16
17     private FileWriter arqW;
18     private BufferedWriter escritor;
19
20     private FileReader arqR;
21     private BufferedReader leitor;
22
23     private List<Pessoa> listaPessoas;
24
25     public String nomeArquivo;
26
27     public Arquivo(String nomeArquivo) {
28         this.nomeArquivo = nomeArquivo;
29         listaPessoas = new ArrayList<>();
30     }
31 }
```

```
33 public List<Pessoa> leArquivo() {
34
35     listaPessoas.clear();
36
37     try {
38         arqR = new FileReader(nomeArquivo + ".txt");
39         leitor = new BufferedReader(arqR);
40
41         String linha;
42
43         while ((linha = leitor.readLine()) != null) {
44
45             String[] campos = linha.split(";");
46
47             String nome = campos[0];
48             char sexo = campos[1].charAt(0);
49             String idioma = campos[2];
50
51             Pessoa p = new Pessoa(nome, sexo, idioma);
52
53             listaPessoas.add(p);
54         }
55
56         leitor.close();
57         arqR.close();
58
59     } catch (FileNotFoundException e) {
60         // Arquivo ainda não existe.
61         // Começa com a lista vazia.
62         System.out.println("Arquivo ainda não existe.");
63
64     } catch (IOException e) {
65         e.printStackTrace();
66     }
67
68     return listaPessoas;
69 }
```

```
72 public List<Pessoa> getListaPessoas() {
73     return listaPessoas;
74 }
75
76 // Grava TODA a lista no arquivo
77 public void gravaArquivo() {
78
79     try {
80         arqW = new FileWriter(nomeArquivo + ".txt", false);
81         escritor = new BufferedWriter(arqW);
82
83         for (Pessoa p : listaPessoas) {
84
85             escritor.write(
86                 p.nome + ";" +
87                 p.sexo + ";" +
88                 p.idioma
89             );
90
91             escritor.newLine();
92         }
93
94         escritor.close();
95         arqW.close();
96
97         System.out.println("Lista salva no arquivo!");
98
99     } catch (IOException e) {
100         e.printStackTrace();
101     }
102 }
103 }
```

```
15 public class Formulario2 extends javax.swing.JFrame {
```

```
17     private static final java.util.logging.Logger logger = java.util.logging.Logger.getLogger(F
```

```
19     /**
```

```
20     * Creates new form Formulario2
```

```
21     */
```

```
22     char sexo;
```

```
23     boolean tecnologia, astronomia, esportes;
```

```
24     private Arquivo arquivo;
```

```
25     private List<Pessoa> listaPessoas;
```

```
26     public Formulario2() {
```

```
27         initComponents();
```

```
28         arquivo = new Arquivo("pessoas");
```

```
29         listaPessoas = arquivo.leArquivo();
```

```
30     }
```

```
31 }
```

```
private void btnSalvarActionPerformed(java.awt.event.ActionEvent evt) {
```

```
    if(rdoMasculino.isSelected()){
```

```
        sexo = 'M';
```

```
    }
```

```
    else if(rdoFeminino.isSelected()){
```

```
        sexo = 'F';
```

```
    }
```

```
    else{
```

```
        JOptionPane.showMessageDialog(null, "Selecione um sexo",
```

```
        "Erro", JOptionPane.ERROR_MESSAGE);
```

```
    }
```

```
    tecnologia = chk_Tecnologia.isSelected();
```

```
    astronomia = chk_Astronomia.isSelected();
```

```
    esportes = chk_Esportes.isSelected();
```

```
    String idioma = cmb_Idioma.getSelectedItem()+"";
```

```
    Pessoa p = new Pessoa (txtNome.getText(), sexo, idioma);
```

```
    listaPessoas.add(p);
```

```
    arquivo.gravaArquivo();
```

```
    System.out.println("Pessoa adicionada!");
```

```
    for (Pessoa pessoa : listaPessoas) {
```

```
        System.out.println(pessoa);
```

```
    }
```

Table

- O JTable em Java é um componente usado para exibir dados em forma de tabela
- Ele permite a criação e exibição de tabelas de dados, semelhantes às que você veria em uma planilha ou em um aplicativo de banco de dados.
- O JTable é altamente personalizável e oferece suporte para uma ampla gama de recursos, como ordenação, seleção de células, edição de células e muito mais.

Table

- Adicione uma tabela:

Nome:

Sexo: ☐ Masculino ☐ Feminino

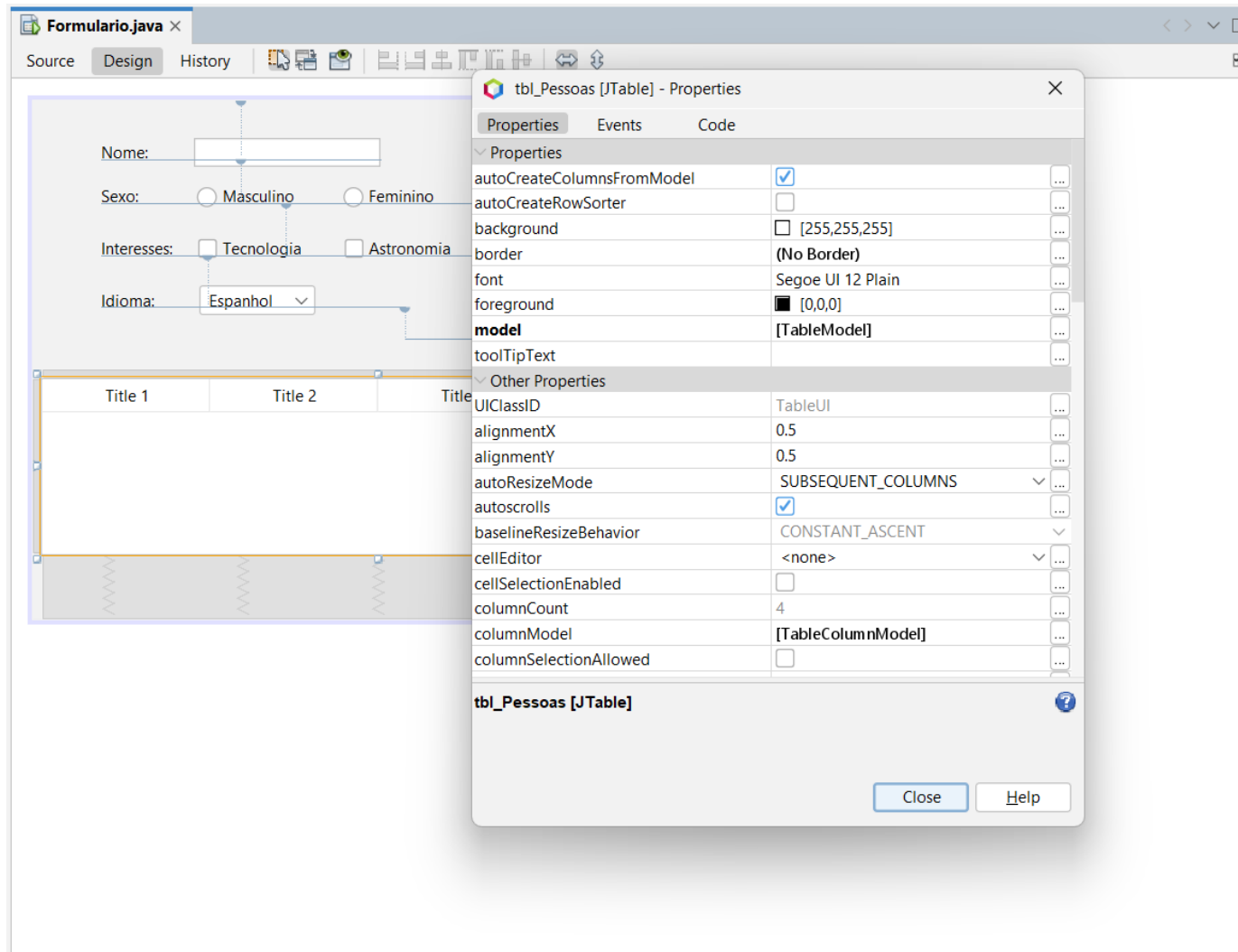
Interesses: ☐ Tecnologia ☐ Astronomia ☐ Esportes

Idioma:

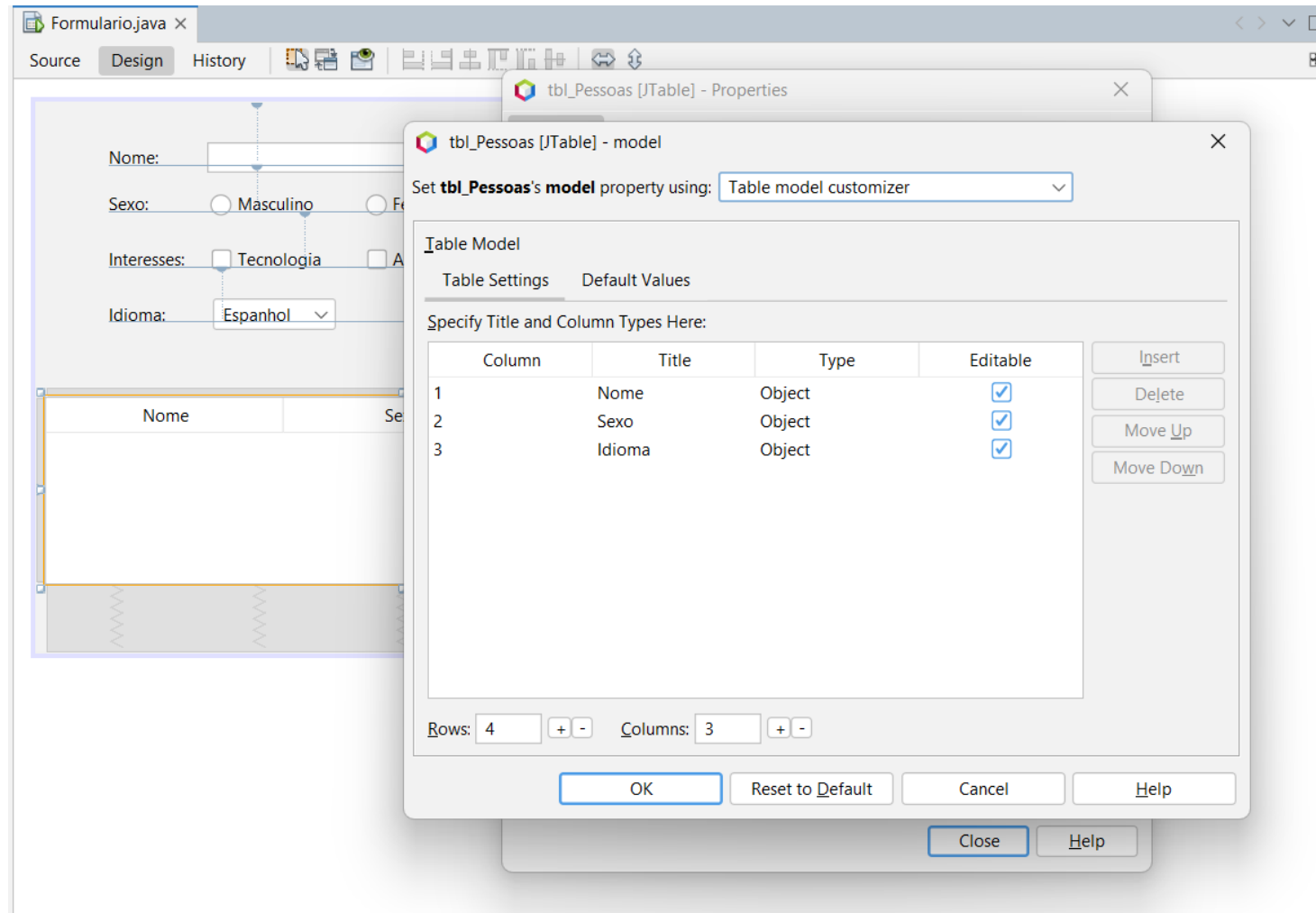
Salvar

Title 1	Title 2	Title 3	Title 4

Table



Table



Table

tbl_Pessoas [JTable] - model

Set **tbl_Pessoas**'s **model** property using: Table model customizer

Table Model

Table Settings Default Values

Specify Title and Column Types Here:

Column	Title	Type	Editable
1	Nome	Object	☒
2	Sexo	Object	☒
3	Idioma	Object	☒

Insert
Delete
Move Up
Move Down

Rows: 0 + - Columns: 3 + -

OK Reset to Default Cancel Help

Table

- Agora vamos adicionar os dados na tabela!
- Para isso, vamos criar um método na classe Pessoa que retornará um objeto com os atributos que irão ser adicionados na tabela
- Adicione o método na classe Pessoa:

```
public Object[] obterDados() {  
    return new Object[] {nome, sexo, idioma};  
}
```

Table

- Na ação do botão Salvar, atualize o código e adicione abaixo do instanciamento do objeto:

```
Pessoa p = new Pessoa(nome: txt_Nome.getText(), sexo, (String) cmb_Idioma.getSelectedItem());
```

```
DefaultTableModel tabela = (DefaultTableModel) tbl_Pessoas.getModel();  
tabela.addRow(rowData: p.obterDados());
```

Table

- Ficou bom?
- O que podemos melhorar?
 - Limpe os dados!
 - Como a gente faz?
 - Aparece todos os elementos da lista?

Table

- Na ação do botão, atualize o código com as linhas abaixo para efetuar a limpeza dos dados.

```
192 Pessoa p = new Pessoa (txtNome.getText(), sexo, idioma);
193
194 listaPessoas.add(p);
195
196 arquivo.gravaArquivo();
197
198 System.out.println("Pessoa adicionada!");
199
200 DefaultTableModel tabela = (DefaultTableModel) tbl_Pessoas.getModel();
201 tabela.addRow(p.obterDados());
202 txtNome.setText("");
203 btnGrpSexo.clearSelection();
204 chk_Tecnologia.setSelected(false);
205 chk_Astronomia.setSelected(false);
206 chk_Esportes.setSelected(false);
207 cmb_Idioma.setSelectedIndex(0);
208
209 }
```

Table

- Ficou bom?
- O que podemos melhorar?
 - Aparece todos os elementos da lista?

Table

```
200      DefaultTableModel tabela = (DefaultTableModel) tbl_Pessoas.getModel();
201      //tabela.addRow(p.obterDados()); agora, vamos buscar direto da lista
202      txtNome.setText("");
203      btnGrpSexo.clearSelection();
204      chk_Tecnologia.setSelected(false);
205      chk_Astronomia.setSelected(false);
206      chk_Esportes.setSelected(false);
207      cmb_Idioma.setSelectedIndex(0);
208
209      tabela.setRowCount(0);
210
211      for (Pessoa pessoa : listaPessoas) {
212          tabela.addRow(pessoa.obterDados());
213      }
214
215  }
```

Table

- Ficou bom?
- O que podemos melhorar?
 - Quando abrimos o programa, já carrega os cadastrados?

Table

```
24  [-] public Formulario2() {
25      initComponents();
26      arquivo = new Arquivo("pessoas");
27      listaPessoas = arquivo.leArquivo();
28      carregarTabela();
29  }
30
31  [-] private void carregarTabela() {
32
33      DefaultTableModel tabela
34          = (DefaultTableModel) tbl_Pessoas.getModel();
35
36      tabela.setRowCount(0);
37
38      [-] for (Pessoa p : listaPessoas) {
39          tabela.addRow(p.obterDados());
40      }
41  }
```

Atividade

- Criar um cadastro de alunos em um sistema
- Cadastro de Aluno:
 - O usuário deve poder adicionar um novo aluno ao sistema. Cada aluno deve ter os seguintes atributos: Nome completo, Data de nascimento (no formato DD/MM/AAAA), Sexo, Matrícula (um número único), Curso, CPF (Cadastro de Pessoa Física), Endereço completo (rua, número, bairro, cidade, CEP), Estado (combo box), Telefone de contato,
- Utilize classes e adicione em uma lista os dados e apresenta na tabela.
- Após, faça a implementação de leitura e escrita em arquivos.
 - Faça com que a lista seja salva em um arquivo texto.
 - Ao abrir o sistema, ele deverá carregar a lista e apresentar na tabela.